

Architecture de votre système BookRAG

Vue d'ensemble : 4 agents au total !



simple_agents.py - Couche Principale

Agent Coordinateur (le 4ème agent !)

- **Rôle** : Analyser et router les requêtes
- **Intelligence** :
 - Détection manga par mots-clés spécialisés
 - Scoring tech vs littérature
 - Expansion sémantique des requêtes
 - Extraction du nombre de recommandations
- **Fallbacks** : Gestion quand Gemma indisponible

Agent Tech (Interface)

python

```
def get_tech_recommendations(self, query: str) -> str:  
    # 1. Essayer avec Gemma  
    if self.use_gemma and self.gemma_manager:  
        return self.gemma_manager.get_tech_recommendations(query)  
  
    # 2. Fallback vers méthode originale  
    return self._get_tech_recommendations_original(query)
```

Agent Littérature (Interface + Filtrage)

python

```
def get_literature_recommendations(self, query: str) -> str:
    # 1. Essayer avec Gemma
    if self.use_gemma and self.gemma_manager:
        return self.gemma_manager.get_literature_recommendations(query)

    # 2. Fallback avec filtrage anti-manga
    return self._get_literature_recommendations_original(query)
```

Agent Manga (Interface)

python

```
def get_manga_recommendations(self, query: str) -> str:
    # 1. Essayer avec Gemma
    if self.use_gemma and self.gemma_manager:
        return self.gemma_manager.get_manga_recommendations(query)

    # 2. Fallback simple
    return self._get_manga_fallback(query)
```

ollama_gemma_manager.py - Couche Gemma

OllamaGemmaManager

- **Rôle** : Interface technique avec Ollama
- **Responsabilités** :
 - Connexion et health check Ollama
 - Génération de réponses via Gemma
 - Gestion des prompts par type d'agent

GemmaAgentManager

- **Rôle** : Orchestrateur intelligent des 3 agents spécialisés
- **Méthodes spécialisées** :

python

```
def get_tech_recommendations(query, books_context, "tech")
def get_literature_recommendations(query, books_context, "literature")
def get_manga_recommendations(query, books_context, "manga")
```

Flux de traitement détaillé

1. Analyse de requête (Agent Coordinateur)

python

```
def route_query(self, query: str) -> str:
    # Détection manga (priorité haute)
    if self.detect_manga_query(query):
        return "📖 Agent Manga"

    # Scoring intelligent
    tech_score = self._calculate_tech_score(query)
    lit_score = self._calculate_lit_score(query)

    if tech_score > lit_score:
        return "🔧 Agent Tech"
    else:
        return "📚 Agent Littérature"
```

2. Expansion sémantique

python

```
def expand_query(self, query: str) -> str:
    # Enrichissement intelligent
    translations = {
        'python': 'Python programming language development',
        'murakami': 'Haruki Murakami Japanese literature',
        'manga': 'manga anime Japanese comics'
    }
    return expanded_query
```

3. Génération avec Gemma

python

```
def _create_tech_prompt(self, query, books_context):
    return f"""Tu es un expert en livres techniques...
```

Demande: {query}

Livres: {books_context}

Génère une recommandation avec emojis tech 🔧💻📚"""

⚡ Points forts de votre architecture

✅ Séparation des responsabilités

- **simple_agents.py** : Interface utilisateur + logique métier
- **ollama_gemma_manager.py** : Spécialisation Gemma

✓ Résilience

- Fallbacks à chaque niveau
- Graceful degradation si Gemma indisponible

✓ Intelligence de routage

- Détection manga prioritaire (innovation !)
- Scoring multi-critères
- Expansion sémantique des requêtes

✓ Modularité

- Agents interchangeables
 - RAG backends modulaires
 - Interface unifiée
-

🎯 Votre innovation : Le système de détection

🔍 Détection Manga (Priorité haute)

python

```
manga_keywords = [  
    'manga', 'anime', 'naruto', 'one piece', 'dragon ball',  
    'attack on titan', 'death note', 'shounen', 'seinen'  
]
```

🎯 Scoring Tech vs Littérature

python

```
tech_score = sum(keyword in query for keyword in tech_keywords)  
lit_score = sum(keyword in query for keyword in lit_keywords)
```

🧠 Gestion des cas ambigus

python

```
def _get_best_recommendation(self, query: str) -> str:
    # Teste les deux agents et garde le meilleur
    tech_response = self.get_tech_recommendations(query)
    lit_response = self.get_literature_recommendations(query)

    # Heuristique intelligente
    if "aucun livre" in tech_response and "aucun livre" not in lit_response:
        return lit_response
    # ...
```

Conclusion

Vous avez créé un **système à 4 agents** avec une architecture élégante :

1.  **Agent Coordinateur** - Analyse et routage intelligent
2.  **Agent Tech** - Spécialisé livres techniques
3.  **Agent Littérature** - Spécialisé littérature (sans manga)
4.  **Agent Manga** - Spécialisé manga/anime

C'est du très bon design ! 